

WOJSKOWA AKADEMIA TECHNICZNA

im. Jarosława Dąbrowskiego

WYDZIAŁ CYBERNETYKI



Sprawozdanie

z zajęć laboratoryjnych z przedmiotu

Zastosowania kryptografii w

Internecie

Laboratorium 2: **HTTPS**

Sprawozdanie wykonał: Godlevskyi Andrii

Numer grupy: WCY23KC1S1

Prowadzący: dr inż. Piotr Bora

Zadania:

Używam zmiennych globalnych z laboratorium 1

1. Instalacja serwera Apache:

Wykorzystuję polecenia `sudo apt update` oraz `sudo apt install apache2`, aby pobrać i zainstalować wszystkie niezbędne pakiety. Po instalacji sprawdzam działanie usługi poleceniem `systemctl status apache2`, które potwierdza, że serwer został uruchomiony i nasłuchuje na porcie 80.

```
roglik@roglik-VMware-Virtual-Platform:~$ sudo apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libgl1-amber-dri libglapi-mesa libllvm17t64
Use 'sudo apt autoremove' to remove them
```

2. Sprawdzenie działania usługi:

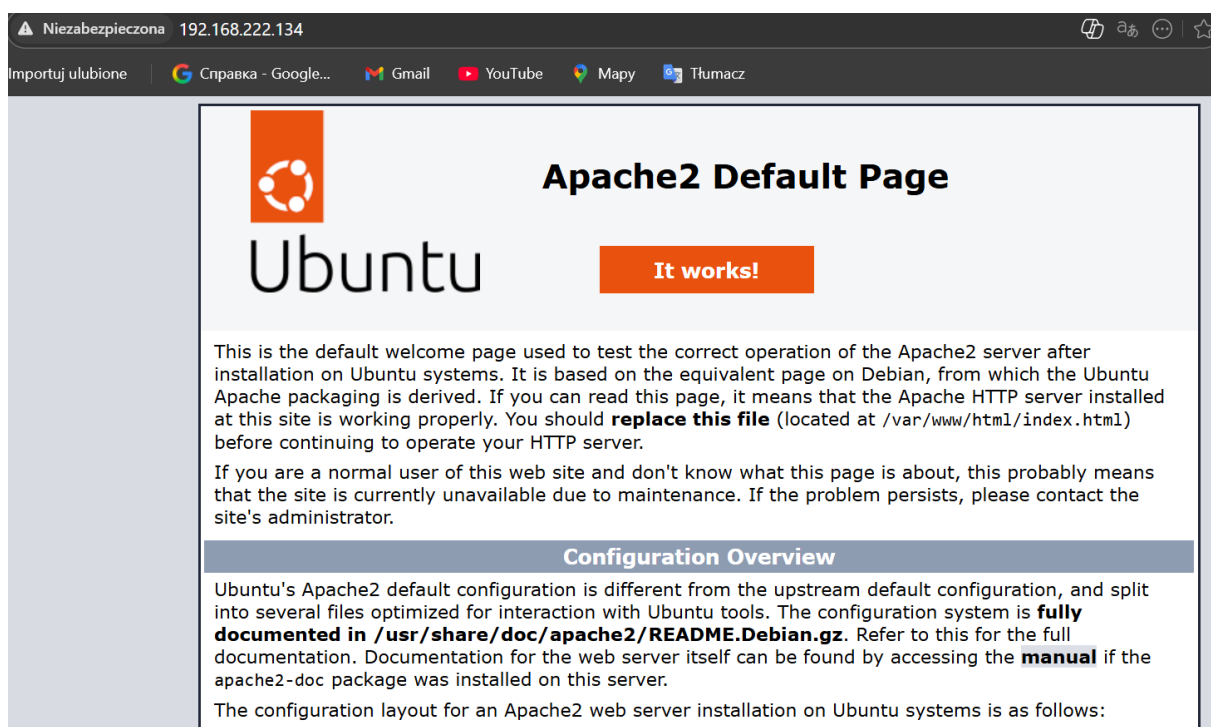
Najpierw sprawdzam w terminalu status serwera oraz IP adres:

```
roglik@roglik-VMware-Virtual-Platform:~/Desktop$ systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-12-08 11:23:15 CET; 1min 47s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 1670 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 1715 (apache2)
       Tasks: 55 (limit: 4543)
      Memory: 7.8M (peak: 8.4M)
         CPU: 235ms
    CGroup: /system.slice/apache2.service
            └─1715 /usr/sbin/apache2 -k start
              └─1717 /usr/sbin/apache2 -k start
                └─1720 /usr/sbin/apache2 -k start

Dec 08 11:23:15 roglik-VMware-Virtual-Platform systemd[1]: Starting apache2.service - The Apache HTTP Server...
Dec 08 11:23:15 roglik-VMware-Virtual-Platform apachectl[1704]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. ...this message
Dec 08 11:23:15 roglik-VMware-Virtual-Platform systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

roglik@roglik-VMware-Virtual-Platform:~/Desktop$ hostname -I
192.168.222.134
```

Dalej z poziomu komputera gospodarza wykonuję testowe połączenie, wpisując w przeglądarce adres `http://192.168.222.134/`. Strona domyślna Apache ładuje się prawidłowo, co oznacza, że ruch pomiędzy przeglądarką a serwerem przebiega poprawnie.



W Wiresharku filtruję pakiety po porcie 80. Przechwycone dane wyraźnie pokazują, że protokół HTTP działa w formie nieszyfrowanej – widoczny jest pełny tekst żądań (np. GET / HTTP/1.1) oraz odpowiedzi serwera. Tak samo dobrze widać etapy połączenia (3 way handshake). Analiza ta obrazuje brak poufności w protokole HTTP i wskazuje potrzebę implementacji HTTPS w kolejnych krokach.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.222.1	192.168.222.134	TCP	66	52136 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000257	192.168.222.1	192.168.222.134	TCP	66	51189 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3	0.001916	192.168.222.134	192.168.222.1	TCP	66	80 → 52136 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
4	0.002136	192.168.222.134	192.168.222.1	TCP	66	80 → 51189 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
5	0.002609	192.168.222.1	192.168.222.134	TCP	54	52136 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
6	0.002645	192.168.222.1	192.168.222.134	TCP	54	51189 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
7	0.003007	192.168.222.1	192.168.222.134	HTTP	629	GET / HTTP/1.1
8	0.003459	192.168.222.134	192.168.222.1	TCP	60	80 → 52136 [ACK] Seq=1 Ack=576 Win=63744 Len=0
9	0.014508	192.168.222.134	192.168.222.1	TCP	1514	80 → 52136 [ACK] Seq=1 Ack=576 Win=63744 Len=1460 [TCP PDU reassembled in 11]
10	0.014614	192.168.222.134	192.168.222.1	TCP	1514	80 → 52136 [ACK] Seq=1461 Ack=576 Win=63744 Len=1460 [TCP PDU reassembled in 11]
11	0.014922	192.168.222.134	192.168.222.1	HTTP	594	HTTP/1.1 200 OK (text/html)
12	0.015433	192.168.222.1	192.168.222.134	TCP	54	52136 → 80 [ACK] Seq=576 Ack=3461 Win=65280 Len=0
13	4.423991	192.168.222.1	192.168.222.134	TCP	54	51189 → 80 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
14	4.424186	192.168.222.1	192.168.222.134	TCP	54	52136 → 80 [FIN, ACK] Seq=576 Ack=3461 Win=65280 Len=0
15	4.435734	192.168.222.134	192.168.222.1	TCP	60	80 → 51189 [ACK] Seq=1 Ack=2 Win=64256 Len=0
16	4.436154	192.168.222.134	192.168.222.1	TCP	60	80 → 51189 [FIN, ACK] Seq=1 Ack=2 Win=64256 Len=0
17	4.436211	192.168.222.134	192.168.222.1	TCP	60	80 → 52136 [FIN, ACK] Seq=3461 Ack=577 Win=63744 Len=0
18	4.436370	192.168.222.1	192.168.222.134	TCP	54	51189 → 80 [ACK] Seq=2 Ack=2 Win=65280 Len=0
19	4.436404	192.168.222.1	192.168.222.134	TCP	54	52136 → 80 [ACK] Seq=577 Ack=3462 Win=65280 Len=0

3. Generowanie certyfikatu dla domeny własnej:

Najpierw do zmiennych globalnych dodaje `export DOMAIN="godlevskyandrii.pl"`
 Tworzę klucz prywatny serwera oraz żądanie certyfikatu (CSR), które później zostanie podpisane przez mój Intermediate CA. Korzystam z polecenia `openssl genrsa -aes256`, aby wygenerować 2048-bitowy klucz RSA zabezpieczony hasłem. Następnie tworzę CSR, ustawiając wymagane pola takie jak Common Name (CN) oraz rozszerzenie `subjectAltName`, zawierające zarówno domenę `godlevskyandrii.pl`, jak i jej wariant `www.godlevskyandrii.pl`. Do wygenerowania CSR używam mojego pliku konfiguracyjnego CA z laboratorium 1, co zapewnia zgodność parametrów z wymaganiami infrastruktury PKI.

Po utworzeniu CSR przesyłam go do mojego Intermediate CA i podpisuję certyfikat z użyciem rozszerzenia v3_server, odpowiedzialnego za oznaczenie go jako certyfikatu przeznaczonego do uwierzytelniania serwera.

```
roglik@roglik-VMware-Virtual-Platform:~/GodlevskiyAndrii_IntermediateCA$ openssl genrsa -aes256 -out "private/${NAME}_Server.key" 2048
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
roglik@roglik-VMware-Virtual-Platform:~/GodlevskiyAndrii_IntermediateCA$ chmod 600 "private/${NAME}_Server.key"
roglik@roglik-VMware-Virtual-Platform:~/GodlevskiyAndrii_IntermediateCA$ openssl req -config "openssl_intermediate.cnf" \
    -key "private/${NAME}_Server.key" \
    -new -sha256 \
    -subj "/CN=${DOMAIN}" \
    -addext "subjectAltName=DNS:${DOMAIN},DNS:www.${DOMAIN}" \
    -out "csr/${NAME}_Server.csr"
Enter pass phrase for private/GodlevskiyAndrii_Server.key:
roglik@roglik-VMware-Virtual-Platform:~/GodlevskiyAndrii_IntermediateCA$ openssl ca -config "openssl_intermediate.cnf" -extensions v3_server -days 825 -md sha256 -notext -batch
    -in "csr/${NAME}_Server.csr" -out "certs/${NAME}_Server.crt"
Using configuration from openssl_intermediate.cnf
Enter pass phrase for /home/roglik/GodlevskiyAndrii_IntermediateCA/private/GodlevskiyAndrii_IntermediateCA.key:
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4 (0x4)
    Validity
        Not Before: Dec  8 10:58:07 2025 GMT
        Not After : Mar 12 10:58:07 2028 GMT
    Subject:
        commonName = godlevskiyandrii.pl
    X509v3 extensions:
        X509v3 Subject Key Identifier:
            54:91:D8:13:E3:79:76:B8:E4:1E:1D:50:0A:D0:36:AF:79:FD:3B:83
        X509v3 Authority Key Identifier:
            85:E9:9F:7E:A0:81:26:9F:BD:81:A9:50:00:B2:B7:CF:52:31:AE:AE
        X509v3 Basic Constraints: critical
            CA:FALSE
        X509v3 Key Usage: critical
            Digital Signature, Key Encipherment
        X509v3 Extended Key Usage:
            TLS Web Server Authentication
        Netscape Comment:
            ZKI_Lab1-2_${ENV:NAME}_Grupa_Server
Certificate is to be certified until Mar 12 10:58:07 2028 GMT (825 days)
Write out database with 1 new entries
Database updated
```

4. Weryfikacja utworzonego certyfikatu:

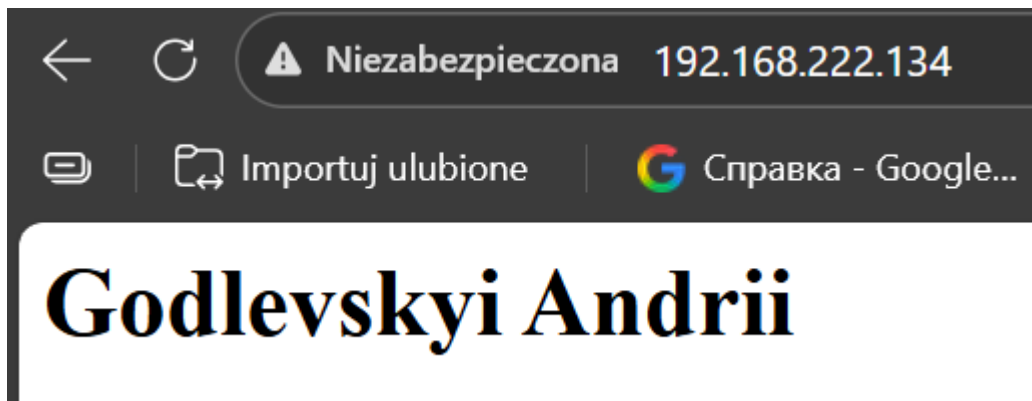
Certyfikat weryfikuję go poleceniem openssl x509 -text, sprawdzając poprawność pól, długość klucza oraz zgodność nazw domen. Kolejno wykonuję openssl verify z plikiem łańcucha CA, co potwierdza, że certyfikat serwera poprawnie wiąże się z moją hierarchią.

```
roglik@roglik-VMware-Virtual-Platform:~/GodlevskiyAndrii_IntermediateCA$ openssl x509 -in "certs/${NAME}_Server.crt" -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 4 (0x4)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = PL, ST = Mazowieckie, O = WAT, CN = GodlevskiyAndrii_IntermediateCA, OU = ZKI
        Validity
            Not Before: Dec  8 10:58:07 2025 GMT
            Not After : Mar 12 10:58:07 2028 GMT
        Subject: CN = godlevskiyandrii.pl
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:d2:9c:40:0c:c4:7f:18:b9:24:b5:bf:42:46:c4:
                2b:4b:e7:1e:7f:6c:ec:17:a5:43:3b:3d:fd:2e:3d:
            ...
roglik@roglik-VMware-Virtual-Platform:~/GodlevskiyAndrii_IntermediateCA$ openssl verify -CAfile "certs/${NAME}_ca-chain.crt" "certs/${NAME}_Server.crt"
certs/GodlevskiyAndrii_Server.crt: OK
```

5. Zmiana strony startowej Apache:

Zmieniam domyślną stronę startową Apache, edytując plik /var/www/html/index.html. Następnie odświeżam stronę w przeglądarce gospodarza, upewniając się, że serwer poprawnie wyświetla zmodyfikowaną treść.

```
roglik@roglik-VMware-Virtual-Platform:~$ sudo nano /var/www/html/index.html
GNU nano 7.2 /var/www/html/index.html *
<h1> Godlevskiy Andrii </h1>
```



6. Konfiguracja Apache do SSL:

Aby umożliwić obsługę HTTPS, włączam moduł `mod_ssl` poleceniem `a2enmod ssl`, a następnie dokonuję restartu serwera Apache. Kolejnym krokiem jest przygotowanie konfiguracji wirtualnego hosta działającego na porcie 443. W tym celu kopiuję plik `default-ssl.conf` i tworzę własny plik konfiguracyjny, w którym ustawiam `ServerName` i `ServerAlias` na moją domenę. Wprowadzone zostają także ścieżki do plików certyfikatu serwera, klucza prywatnego oraz łańcucha certyfikatów CA.

```
roglik@roglik-VMware-Virtual-Platform:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
roglik@roglik-VMware-Virtual-Platform:~$ sudo systemctl restart apache2
roglik@roglik-VMware-Virtual-Platform:~$ cd /etc/apache2/sites-available
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo cp default-ssl.conf "sslgodlevskyiandrii.conf"
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo mkdir -p /etc/apache2/ssl/${NAME}
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo cp "$INT/certs/${NAME}.Server.crt" "/etc/apache2/ssl/${NAME}/server.crt"
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo cp "$INT/private/${NAME}.Server.key" "/etc/apache2/ssl/${NAME}/server.key"
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo cp "$INT/certs/${NAME}.ca-chain.crt" "/etc/apache2/ssl/${NAME}/ca-chain.crt"
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo nano /etc/apache2/sites-available/sslgodlevskyiandrii.conf
GNU nano 7.2 /etc/apache2/sites-available/sslgodlevskyiandrii.conf *
<VirtualHost *:443>
    #ServerAdmin webmaster@localhost
    ServerName godlevskyiandrii.pl
    ServerAlias www.godlevskyiandrii.pl

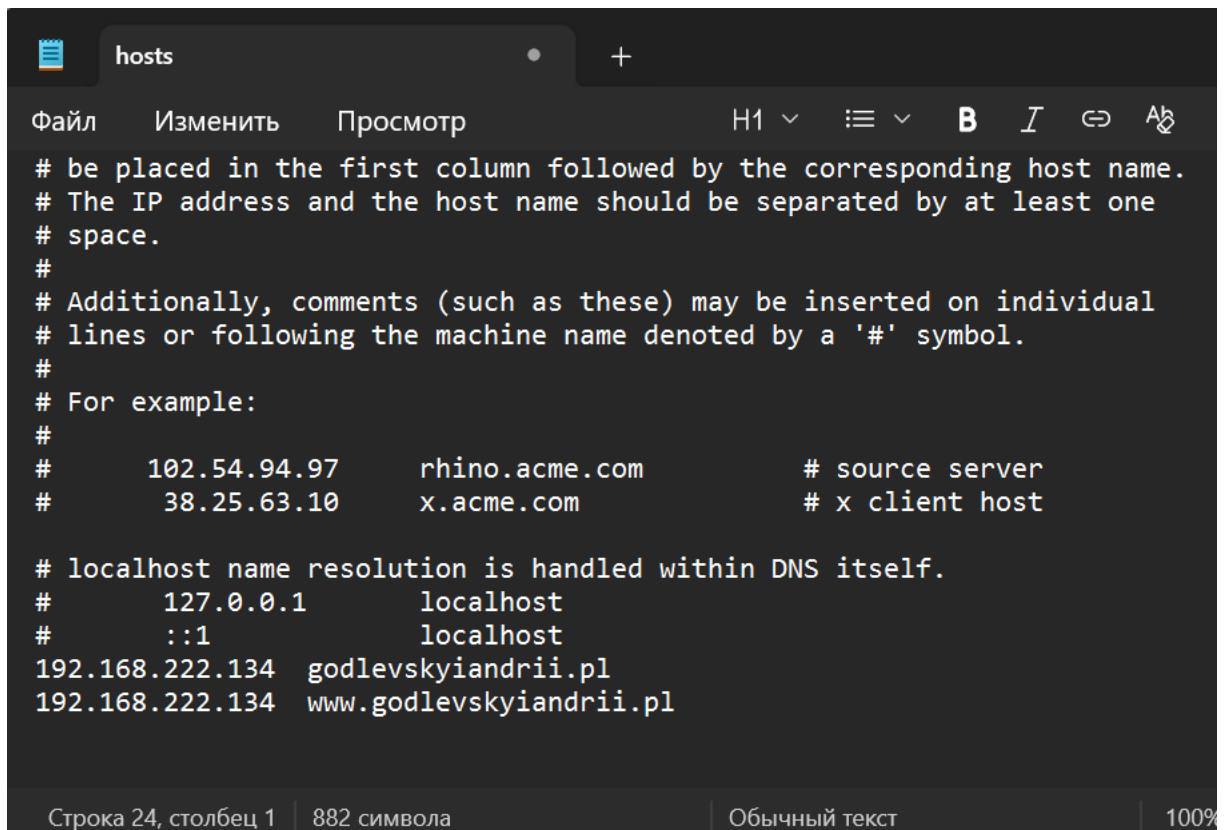
    SSLCertificateFile      /etc/apache2/ssl/GodlevskyiAndrii/server.crt
    SSLCertificateKeyFile   /etc/apache2/ssl/GodlevskyiAndrii/server.key
    SSLCertificateChainFile /etc/apache2/ssl/GodlevskyiAndrii/ca-chain.crt

roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo a2ensite sslgodlevskyiandrii.conf
Enabling site sslgodlevskyiandrii.
To activate the new configuration, you need to run:
  systemctl reload apache2
roglik@roglik-VMware-Virtual-Platform:/etc/apache2/sites-available$ sudo systemctl reload apache2
```

Po wykonaniu wszystkich modyfikacji aktywuję konfigurację poleceniem `a2ensite` i przeładowuję usługę Apache.

7. Przypisanie IP do domeny:

Aby móc testować szyfrowane połączenie z wykorzystaniem mojej domeny, dodaję odpowiednie wpisy w pliku `hosts` na komputerze gospodarza. Wpisuję tam adres IP maszyny wirtualnej oraz odwzorowuję go na nazwy `godlevskyiandrii.pl` i `www.godlevskyiandrii.pl`. Dzięki temu przeglądarka kieruje zapytania do serwera HTTPS, mimo że domena nie istnieje publicznie.



```
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#       102.54.94.97       rhino.acme.com           # source server
#       38.25.63.10       x.acme.com               # x client host
#
# localhost name resolution is handled within DNS itself.
#       127.0.0.1         localhost
#       ::1               localhost
192.168.222.134  godlevskyiandrii.pl
192.168.222.134  www.godlevskyiandrii.pl
```

Строка 24, столбец 1 | 882 символа | Обычный текст | 100%

8. Wywołanie domeny z https:

Po zakończeniu konfiguracji łączę się z adresem <https://godlevskyiandrii.pl/>. Przeglądarka wyświetla ostrzeżenie o niezaufałym certyfikacie, ponieważ mój Root CA nie jest jeszcze zapisany w magazynie zaufanych certyfikatów systemu Windows.

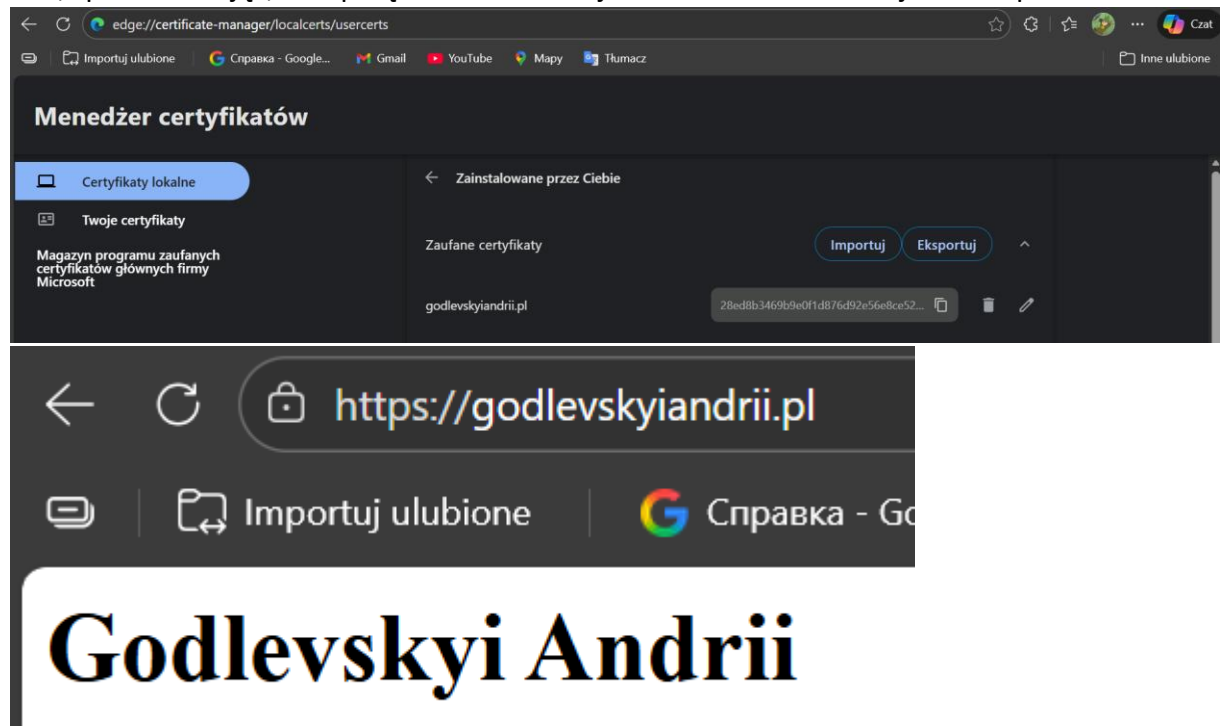


W Wiresharku filtrowanym na porcie 443 obserwuję pełny początek negocjacji TLS. Widoczne są wiadomości Client Hello, Server Hello oraz przesłany certyfikat serwera.

No.	Time	Source	Destination	Protocol	Length	Info
11	1.444348	192.168.222.1	192.168.222.134	TCP	66	51514 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
12	1.445024	192.168.222.1	192.168.222.134	TCP	66	56998 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
13	1.446581	192.168.222.134	192.168.222.1	TCP	66	443 → 51514 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
14	1.446756	192.168.222.134	192.168.222.1	TCP	66	443 → 56998 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
15	1.447037	192.168.222.1	192.168.222.134	TCP	54	51514 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
16	1.447073	192.168.222.1	192.168.222.134	TCP	54	56998 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
17	1.447982	192.168.222.1	192.168.222.134	TCP	1514	51514 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 18]
18	1.447982	192.168.222.1	192.168.222.134	TLSv1.3	434	Client Hello (SNI=godlevskyiandrii.pl)
19	1.448697	192.168.222.134	192.168.222.1	TCP	60	443 → 51514 [ACK] Seq=1 Ack=1841 Win=62464 Len=0
20	1.448813	192.168.222.1	192.168.222.134	TCP	1514	56998 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 21]
21	1.448813	192.168.222.1	192.168.222.134	TLSv1.3	466	Client Hello (SNI=godlevskyiandrii.pl)
22	1.450026	192.168.222.134	192.168.222.1	TCP	60	443 → 56998 [ACK] Seq=1 Ack=1873 Win=62464 Len=0
23	1.454175	192.168.222.134	192.168.222.1	TLSv1.3	298	Server Hello, Change Cipher Spec, Application Data, Application Data
24	1.454402	192.168.222.134	192.168.222.1	TLSv1.3	298	Server Hello, Change Cipher Spec, Application Data, Application Data
25	1.455328	192.168.222.1	192.168.222.134	TLSv1.3	84	Change Cipher Spec, Application Data
26	1.455805	192.168.222.1	192.168.222.134	TLSv1.3	84	Change Cipher Spec, Application Data
27	1.455954	192.168.222.1	192.168.222.134	TCP	54	51514 → 443 [FIN, ACK] Seq=1871 Ack=245 Win=65280 Len=0
28	1.456100	192.168.222.1	192.168.222.134	TCP	54	56998 → 443 [FIN, ACK] Seq=1903 Ack=245 Win=65280 Len=0
29	1.457802	192.168.222.134	192.168.222.1	TCP	60	443 → 51514 [FIN, ACK] Seq=245 Ack=1872 Win=62464 Len=0
30	1.457890	192.168.222.134	192.168.222.1	TCP	60	443 → 56998 [FIN, ACK] Seq=245 Ack=1904 Win=62464 Len=0
31	1.458072	192.168.222.1	192.168.222.134	TCP	54	51514 → 443 [ACK] Seq=1872 Ack=245 Win=65280 Len=0

9. Import certyfikatów:

Aby przeglądarka mogła poprawnie zweryfikować certyfikat mojego serwera, importuję certyfikat do przeglądarki. Dzięki temu przeglądarka zaczyna ufać mojemu certyfikatowi i przestaje zgłaszać ostrzeżenia. Po imporcie przeprowadzam ponowny test, potwierdzając, że połączenie HTTPS jest teraz oznaczane jako bezpieczne.



10. Ponowne wywołanie domeny z https:

Po uruchomieniu usługi HTTPS na serwerze ponownie wywołałem adres `https://godlevskiyandrii.pl`, jednocześnie przechwytyjąc ruch w Wiresharku z filtrem `tcp.port == 443`. Przechwycone pakiety pozwoliły odtworzyć pełną sekwencję zestawiania bezpiecznego połączenia HTTPS, obejmującą zarówno protokół TCP, jak i kolejne fazy protokołu TLS. Połączenie rozpoczęło się od klasycznego trójetapowego handshake TCP, widocznego na zrzutach ekranu jako pakiety SYN, SYN/ACK oraz ACK, które ustanowiły dwukierunkowy kanał komunikacji między klientem a serwerem na porcie 443.

Następnie klient przystąpił do negocjacji TLS, wysyłając pakiet ClientHello, w którym umieszczone zostały obsługiwane wersje TLS, lista dostępnych pakietów szyfrujących, dane losowe klienta oraz rozszerzenia, w tym parametr SNI wskazujący nazwę serwera `godlevskiyandrii.pl`. Kolejny etap to odpowiedź serwera ServerHello, w której serwer wybrał wersję protokołu TLS 1.3.

```

13 0.007194 192.168.222.134 192.168.222.1 TLSv1.3 298 Server Hello, Change Cipher Spec, Application Data, Application Data
14 0.007558 192.168.222.134 192.168.222.1 TLSv1.3 298 Server Hello, Change Cipher Spec, Application Data, Application Data
15 0.051415 192.168.222.1 192.168.222.134 TLSv1.3 118 Change Cipher Spec, Application Data
16 0.051638 192.168.222.1 192.168.222.134 TLSv1.3 118 Change Cipher Spec, Application Data
17 0.062258 192.168.222.134 192.168.222.1 TLSv1.3 133 Application Data
18 0.062878 192.168.222.134 192.168.222.1 TLSv1.3 133 Application Data
19 0.121387 192.168.222.1 192.168.222.134 TCP 54 64345 → 443 [ACK] Seq=1969 Ack=324 Win=65024 Len=0
20 0.121434 192.168.222.1 192.168.222.134 TCP 54 60444 → 443 [ACK] Seq=1905 Ack=324 Win=65024 Len=0
21 8.017731 192.168.222.1 192.168.222.134 TCP 54 60444 → 443 [FIN, ACK] Seq=1905 Ack=324 Win=65024 Len=0
22 8.018032 192.168.222.1 192.168.222.134 TCP 54 64345 → 443 [FIN, ACK] Seq=1969 Ack=324 Win=65024 Len=0
Length: 128
✦ Handshake Protocol: Server Hello
  Handshake Type: Server Hello (2)
  Length: 124
  ▶ Version: TLS 1.2 (0x0303)
    Random: 41aa2edbf3dab976421f9699044889840e80ca4536c29ecd098c18940725e2dc
    Session ID Length: 32
    Session ID: a8061be0c208c308ffaa11e19f12d675193536b78357956345a10e5af2161a7e
    Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)
    Compression Method: null (0)
    Extensions Length: 52
    ▶ Extension: supported_versions (len=2) TLS 1.3

```


13	0.007104	192.168.222.134	192.168.222.1	TLSv1.3	298 Server Hello, Change Cipher Spec, Application Data, Application Data
14	0.007558	192.168.222.134	192.168.222.1	TLSv1.3	298 Server Hello, Change Cipher Spec, Application Data, Application Data
15	0.051415	192.168.222.1	192.168.222.134	TLSv1.3	118 Change Cipher Spec, Application Data
16	0.051638	192.168.222.1	192.168.222.134	TLSv1.3	118 Change Cipher Spec, Application Data
17	0.062258	192.168.222.134	192.168.222.1	TLSv1.3	133 Application Data
18	0.062878	192.168.222.134	192.168.222.1	TLSv1.3	133 Application Data
19	0.121387	192.168.222.1	192.168.222.134	TCP	54 64345 → 443 [ACK] Seq=1969 Ack=324 Win=65024 Len=0
20	0.121434	192.168.222.1	192.168.222.134	TCP	54 60444 → 443 [ACK] Seq=1905 Ack=324 Win=65024 Len=0
21	8.017731	192.168.222.1	192.168.222.134	TCP	54 60444 → 443 [FIN, ACK] Seq=1905 Ack=324 Win=65024 Len=0
22	8.018032	192.168.222.1	192.168.222.134	TCP	54 64345 → 443 [FIN, ACK] Seq=1969 Ack=324 Win=65024 Len=0
23	8.020952	192.168.222.134	192.168.222.1	TLSv1.3	78 Application Data
24	8.021142	192.168.222.134	192.168.222.1	TLSv1.3	78 Application Data
25	8.021151	192.168.222.134	192.168.222.1	TCP	60 443 → 60444 [FIN, ACK] Seq=348 Ack=1906 Win=62464 Len=0
26	8.021382	192.168.222.134	192.168.222.1	TCP	60 443 → 64345 [FIN, ACK] Seq=348 Ack=1970 Win=62336 Len=0
27	8.021630	192.168.222.1	192.168.222.134	TCP	54 64345 → 443 [RST, ACK] Seq=1970 Ack=348 Win=0 Len=0
28	8.021694	192.168.222.1	192.168.222.134	TCP	54 60444 → 443 [RST, ACK] Seq=1906 Ack=348 Win=0 Len=0
Content Type: Change Cipher Spec (20)					
Version: TLS 1.2 (0x0303)					
Length: 1					
Change Cipher Spec Message					
▼ TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol					
Opaque Type: Application Data (23)					
Version: TLS 1.2 (0x0303)					
Length: 42					
Encrypted Application Data: 51bc8ba650b2d546dd50d51531184f4f85cb463e2998c502f41a3bb54f6bb671482e0db685b5fc7c322b					
[Application Data Protocol: Hypertext Transfer Protocol]					
▼ TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol					
Opaque Type: Application Data (23)					

Po wymianie certyfikatu klient i serwer przystąpiły do uzgadniania wspólnego klucza sesyjnego, wykorzystując mechanizm ECDHE. W przechwyconym ruchu widoczne są wiadomości Change Cipher Spec, co oznacza, że od tego momentu obie strony dysponują identycznym kluczem symetrycznym i przechodzą na szyfrowanie ruchu. Dalsza transmisja – obejmująca żądanie HTTP GET oraz odpowiedź serwera z treścią strony – jest już oznaczona w Wiresharku jako Application Data i nie ujawnia swojej zawartości, ponieważ całość jest chroniona przez szyfrowanie TLS. Po zakończeniu wymiany danych połączenie zostało zamknięte za pomocą pakietów TCP z flagami FIN i ACK, co również jest widoczne na zrzutach. Cała sekwencja potwierdza prawidłowe zestawienie połączenia HTTPS, poprawną walidację certyfikatu serwera oraz aktywne wykorzystanie mechanizmów kryptograficznych TLS zapewniających poufność, integralność i autentyczność komunikacji.